Tutorial : Restaurant Data

Follow the instructions below to set up your database and complete the exercises. These tasks are designed to test your understanding of NoSQL data retrieval, aggregation pipelines, and pattern matching as discussed in the lectures.

1. Data Setup

Copy and paste the following command into your MongoDB shell to create the restaurants collection with sample data:

```
db.restaurants.insertMany([
  {
    "name": "Morris Park Bake Shop",
    "borough": "Bronx",
    "cuisine": "Bakery",
    "restaurant_id": "30075445",
    "address": { "street": "Morris Park Ave", "zipcode": "10462" },
    "grades": [
      { "grade": "A", "score": 2 },
      { "grade": "A", "score": 6 },
      { "grade": "A", "score": 10 },
      { "grade": "B", "score": 14 }
    ]
  },
  {
    "name": "Empire State Bakery",
    "borough": "Manhattan",
    "cuisine": "Bakery",
    "restaurant_id": "40012345",
    "address": { "street": "5th Ave", "zipcode": "10001" },
    "grades": [
      { "grade": "A", "score": 20 },
      { "grade": "C", "score": 35 }
    ]
  },
  {
    "name": "Bronx Delights",
    "borough": "Bronx",
    "cuisine": "Deli",
    "restaurant_id": "50067890",
    "address": { "street": "Main St", "zipcode": "10451" },
    "grades": [
      { "grade": "A", "score": 5 },
      { "grade": "B", "score": 12 }
    ]
  }
]);
```

2. Exercises Questions

1. Find all restaurants in the "Bronx" that have received a grade of "A" with a score of exactly 10 in the same inspection record. Hint: Use the $elemMatch operator to ensure both conditions match the same object inside the grades array.

```
db.restaurants.find({
  "borough": "Bronx",
  "grades": {
    $elemMatch: { "grade": "A", "score": 10 }
  }
})
```

```
week4> db.restaurants.find({ borough: "Bronx", grades: { $elemMatch: { grade: 'A', score: 10 } } })
[
  {
    _id: ObjectId('69c0a08e4a35f23724c4fce6'),
    name: 'Morris Park Bake Shop',
    borough: 'Bronx',
    cuisine: 'Bakery',
    restaurant_id: '30075445',
    address: { street: 'Morris Park Ave', zipcode: '10462' },
    grades: [
      { grade: 'A', score: 2 },
      { grade: 'A', score: 6 },
      { grade: 'A', score: 10 },
      { grade: 'B', score: 14 }
    ]
  }
]
week4>
```

2. Create an aggregation pipeline to find the total number of restaurants in each borough that have a cuisine of "Bakery". Hint: Use a $match stage followed by a $group stage. In the $group stage, use _id: "$borough" and the $sum accumulator.

```
db.restaurants.aggregate([
  { $match: { "cuisine": "Bakery" } },
  { $group: { _id: "$borough", totalRestaurants: { $sum: 1 } } }
])
```

```
week4> db.restaurants.aggregate([ { $match: {"cuisine": "Bakery"}}, { $group: { _id: "$borough", resturantCount: { $sum: 1}}}])
[
  { _id: 'Manhattan', resturantCount: 1 },
  { _id: 'Bronx', resturantCount: 1 }
]
```

3. Find all restaurants whose name contains the word "Bakery" or "Bake". The search must be case-insensitive. Hint: Use the $regex operator with the i option.

```
db.restaurants.find({
  "name": { $regex: /Bakery|Bake/i }
})
```

```
week4> db.restaurants.find({
| name: {$regex: /Bakery|bake/i}
| })
 [
   {
     _id: ObjectId('69c0a08e4a35f23724c4fce6'),
     name: 'Morris Park Bake Shop',
     borough: 'Bronx',
     cuisine: 'Bakery',
     restaurant_id: '30075445',
     address: { street: 'Morris Park Ave', zipcode: '10462' },
     grades: [
       { grade: 'A', score: 2 },
       { grade: 'A', score: 6 },
       { grade: 'A', score: 10 },
       { grade: 'B', score: 14 }
     ]
   },
   {
     _id: ObjectId('69c0a08e4a35f23724c4fce7'),
     name: 'Empire State Bakery',
     borough: 'Manhattan',
     cuisine: 'Bakery',
     restaurant_id: '40012345',
     address: { street: '5th Ave', zipcode: '10001' },
     grades: [ { grade: 'A', score: 20 }, { grade: 'C', score: 35 } ]
   }
 ]
 week4>
```

4. Retrieve all restaurants in the "Bronx". In the result, only show the restaurant name and the street from the nested address object. Hide the _id field. Hint: Use Dot Notation (e.g., "address.street") in your projection object.

```
db.restaurants.find(
  { "borough": "Bronx" },
  { "name": 1, "address.street": 1, "_id": 0 }
)
```

```
week4> db.restaurants.find( {borough : "Bronx"}, {name: 1, "address.street":1 } )
[
  {
    _id: ObjectId('69c0a08e4a35f23724c4fce6'),
    name: 'Morris Park Bake Shop',
    address: { street: 'Morris Park Ave' }
  },
  {
    _id: ObjectId('69c0a08e4a35f23724c4fce8'),
    name: 'Bronx Delights',
    address: { street: 'Main St' }
  }
]
week4> db.restaurants.find( {borough : "Bronx"}, {name: 1, "address.street":1, _id : 0 } )
[
  {
    name: 'Morris Park Bake Shop',
    address: { street: 'Morris Park Ave' }
  },
  { name: 'Bronx Delights', address: { street: 'Main St' } }
]
week4>
```

5. Use the aggregation pipeline to list the names of all "Bakery" restaurants, sorted by their name in descending order. Hint: Start with a $match stage for the cuisine, then use a $sort stage with a value of -1.

```
db.restaurants.aggregate([
  { $match: { "cuisine": "Bakery" } },
  { $sort: { "name": -1 } }
])
```

```
week4> db.restaurants.aggregate([ {$match: {"borough": "Bronx"}}, {$sort: {"name" : -1}} ])
[
  {
    _id: ObjectId('69c0a08e4a35f23724c4fce6'),
    name: 'Morris Park Bake Shop',
    borough: 'Bronx',
    cuisine: 'Bakery',
    restaurant_id: '30075445',
    address: { street: 'Morris Park Ave', zipcode: '10462' },
    grades: [
      { grade: 'A', score: 2 },
      { grade: 'A', score: 6 },
      { grade: 'A', score: 10 },
      { grade: 'B', score: 14 }
    ]
  },
  {
    _id: ObjectId('69c0a08e4a35f23724c4fce8'),
    name: 'Bronx Delights',
    borough: 'Bronx',
    cuisine: 'Deli',
    restaurant_id: '50067890',
    address: { street: 'Main St', zipcode: '10451' },
    grades: [ { grade: 'A', score: 5 }, { grade: 'B', score: 12 } ]
  }
]
```